

1. **What REST is:** What REST is · Stateless requests
2. **Designing URLs:** Naming resources: nouns, plurals, ids and nesting · Filters, sorting and pagination: the query string · Versioning: changing an API without breaking its clients
3. **Methods and CRUD:** CRUD on HTTP: the routes of a resource · Safe and idempotent: why retries matter · PUT replaces, PATCH changes
4. **Status codes and errors:** Success codes: 200, 201 and 204 · Client errors: choosing the right 4xx · 401, 403 or 404: who are you, and may you? · One error shape, and when to use 500
5. **The API contract:** JSON in, JSON out: Content-Type and Accept · The API contract vs the stored record (DTOs)
6. **Building and testing:** CRUD with an in-memory array · Validating input: 400 with field errors · Testing by hand: curl, Postman, Thunder Client · Automated API tests with supertest
7. **Documenting and other API styles:** Documenting the API: OpenAPI · REST, GraphQL and other API styles

What REST is

What REST is

REST (Representational State Transfer) is a way of designing HTTP APIs: every thing the API knows about has its own URL, and the same few HTTP methods read and change all of them.

REST vocabulary

Term	Meaning	Example
Resource	A thing the API lets you name with a URL	a task, a user, the list of tasks
Collection	The resource that holds many items	/api/tasks
Item	One resource inside a collection	/api/tasks/7
Representation	The resource written in a format and sent over HTTP: not the stored record, a description of it	{"id":7,"title":"Study","done":false}
Endpoint	One method + path pair the API supports	DELETE /api/tasks/:id

The rules that shape everyday design

- Client-server** The front end and the back end are separate programs that only talk over HTTP.
- Uniform interface** Methods, status codes and headers mean the same on every resource: DELETE always removes, 404 always means "not there".
- Stateless** Every request carries everything needed to answer it (see [Stateless requests](#)).
- Cacheable** A GET answer may be reused.

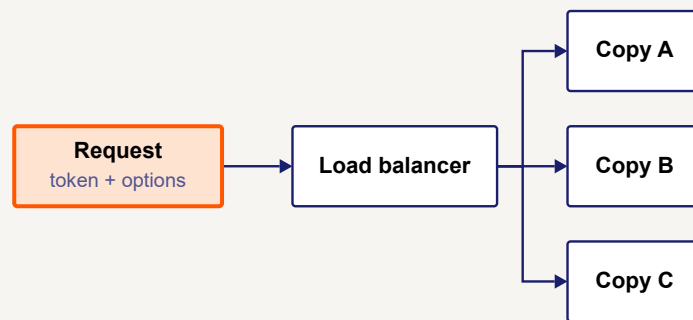
Example. The same features designed twice. Action style: POST /getTask with {"id":7}, POST /markTaskDone?id=7, POST /removeTask?id=7. REST style: GET /api/tasks/7, PATCH /api/tasks/7 with {"done":true}, DELETE /api/tasks/7. A newcomer who has seen GET /api/users/3 guesses DELETE /api/users/3; the action version needs the documentation for every call.

Common mistake. Believing an API is RESTful because it answers JSON. POST /api/doEverything?action=delete&id=7 answers JSON and breaks every rule: the URL names an action, the method means nothing and the status code says nothing. REST is about resources and a uniform interface, not the format.

Stateless requests

In a **stateless API** each request carries everything the server needs to handle it (who is calling, which

resource, which options), and the server keeps no memory of that client between requests.



The request carries the context, so it does not matter which copy answers.

Resource state: stored

The tasks themselves. They are the point of the API and they persist, in memory now and in a database later.

Conversation state: not in the server

"This client is logged in", "this client is on page 2", "this client chose done=false". It travels with each request instead: a token in the Authorization header, ?page=2&done=false in the URL.

```

// X Stateful: the server keeps "who is logged in" in its own memory
let currentUser = null;
app.post('/api/login', (req, res) => {
  // shared by EVERY client of this process
  currentUser = req.body.email;
  res.sendStatus(204);
});
app.get('/api/my-books', (req, res) => {
  res.json(books.filter((b) => b.owner === currentUser));
});

// ✓ Stateless: every request says who it comes from
app.get('/api/my-books', requireToken, (req, res) => {
  // req.user set from the token
  res.json(books.filter((b) => b.owner === req.user.email));
});
  
```

Example. With the stateful code, Ana logs in, then Leo logs in: `currentUser` is now Leo, and Ana's next GET `/api/my-books` returns **Leo's** books. With two copies it gets worse: Ana logged in on copy A, her next request lands on copy B, where `currentUser` is null. In the stateless version each request carries `Authorization: Bearer <token>`, so who logged in last, or which copy answers, no longer matters.

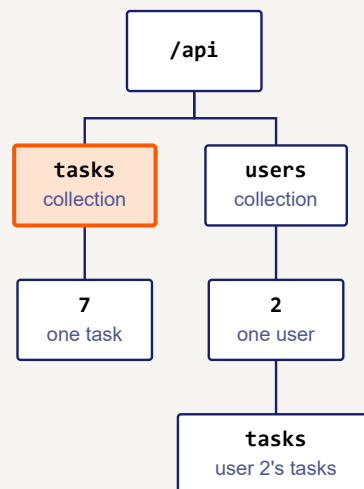
Common mistake. Thinking "stateless" forbids a database, or forbids remembering a logged-in user. The data lives in storage (resource state), and the **client** remembers being logged in by sending its token every time. Only per-client memory **inside the server process** is ruled out.

Designing URLs

Naming resources: nouns, plurals, ids and nesting

A RESTful URL names a **thing**, never an action: a plural noun for the collection (`/api/tasks`), the collection plus

an id for one item (/api/tasks/7), and a nested path when a resource belongs to another (/api/users/2/tasks).



Paths read like folders: a collection, its items, and at most one level of nesting.

- **One plural noun per resource**, for both levels: /api/tasks and /api/tasks/7, never /api/task/7.
- **The id goes in the path**, because it identifies the resource; the query string is for options (see [Filters, sorting and pagination](#)).
- **Lower case with hyphens**: /api/study-groups. Paths are case-sensitive, so /api/Tasks is another URL.
- **An /api prefix** keeps the API apart from pages the same server may serve.

From action URLs to resource URLs

Instead of	Write	Why
GET /api/getAllTasks	GET /api/tasks	The method already says "get"
POST /api/createTask	POST /api/tasks	Create = POST on the collection
GET /api/tasks?id=7	GET /api/tasks/7	Identity belongs in the path
POST /api/tasks/7/delete	DELETE /api/tasks/7	The method is the action
PUT /api/tasks/7/markDone	PATCH /api/tasks/7 + {"done":true}	Changing a field is an update
GET /api/tasksOfUser?user=2	GET /api/users/2/tasks OR ?userId=2	Nesting or a filter, not a new name

Example. A book club API: GET /api/books (all books), GET /api/books/12 (one book), POST /api/books (add one), GET /api/books/12/reviews (the reviews of book 12), POST /api/books/12/reviews (review it), DELETE /api/reviews/88 (one review, by its own id). Six endpoints, two nouns, and every method means what it always means.

Common mistake. Adding a verb "because it is clearer", such as GET /api/books/search. Now /api/books/search collides with /api/books/:id (Express may read search as an id) and clients learn one more name. A search is a filter on the collection: GET /api/books?search=dune.

Filters, sorting and pagination: the query string

The path says **which resource**; the query string says **which part of it, in what order and how much**: **filters** (?done=false), **search** (?search=css), **sorting** (?sort=createdAt&order=desc) and **pagination** (?page=2&limit=10).

The pagination contract

Request	?page=2&limit=10: items 11–20. The same window as offset style, ?offset=10&limit=10: offset = (page – 1) × limit.
Response body	A plain JSON array of the items on that page, so GET /api/tasks stays a list.

- Response headers** X-Total-Count: 57 (how many match in total) and, optionally, Link: </api/tasks?page=3&limit=10>; rel="next".
- Order** Always sort by something stable, with a tie-breaker (createdAt, then id), or items move between pages.
- Limits** Default the page size and cap it (at most 100, say): someone will try ? limit=1000000 (see [Limits that protect the API](#)).

- **Filter on the server**, never by sending everything to the browser: it wastes bandwidth and can expose data the user should not see.
- **Bad parameters**: ignore unknown names (?colour=red); reject malformed values (?limit=abc, ?done=maybe) with **400**, one error per parameter.
- **No matches is not an error**: 200 with [], never 404.

```
// req.query in Express holds the same thing: every value is a STRING
const params = new URLSearchParams('done=false&page=2&limit=500&tag=a&tag=b');

console.log(params.get('done'), typeof params.get('done'));
// the string "false" is truthy!
console.log(Boolean(params.get('done')));
// the right test
console.log(params.get('done') === 'true');

const page = Math.max(1, Number(params.get('page')) || 1);
// clamped
const limit = Math.min(100, Number(params.get('limit')) || 20);
console.log({ page, limit, offset: (page - 1) * limit });

// repeated keys
console.log(params.get('tag'), params.getAll('tag'));
```

Example. Five tasks, of which 2, 3 and 4 are unfinished. GET /api/tasks?done=false&sort=createdAt&order=desc&limit=2 returns 200 with tasks 4 and 3 and X-Total-Count: 3, so the client knows a second page exists. GET /api/tasks?done=false&page=5&limit=2 returns 200 with []: the page is past the end, not "not found".

Common mistake. Paging without a stable order. Sorted only by createdAt, two tasks created in the same millisecond can swap places between requests: one appears on page 1 and page 2, the other on neither. Add a tie-breaker that is unique, such as the id.

Versioning: changing an API without breaking its clients

A version in the URL (/api/v1/tasks) or in a header lets you publish an incompatible change as a new version while existing clients keep using the old one.

Which changes break clients

Kind	Examples	New version?
Non-breaking	A new endpoint, a new optional query parameter, a new field in a response	No
Breaking	Renaming or removing a field; changing a type (done from a boolean to "yes"); making an optional field required; changing a URL or a status code	Yes

In the path /api/v1/tasks, mounted with app.use('/api/v1/tasks', tasksRouterV1). Visible, easy to try with curl, easy to route. The most common choice.

In a header Accept: application/vnd.example.v2+json. Clean URLs, but harder to try in a browser.

Deprecated When v2 ships, v1 keeps working for a while, announced as scheduled to be switched off.

Example. Version 1 returns { "id": 7, "name": "Study" } and the field should be called `title`. Renaming it would crash every client that reads `task.name`. Instead: in v1 send **both** `name` and `title` (adding is non-breaking); publish `/api/v2/tasks` with only `title`; mark v1 deprecated; switch it off once the clients have moved.

Common mistake. Renaming a field "to tidy up" inside the same version, or creating a version for every small addition (`/v1 ... /v14`). Additions need no version; only breaking changes do.

Methods and CRUD

CRUD on HTTP: the routes of a resource

CRUD (Create, Read, Update, Delete) names the four things almost every API does to its data; REST maps them onto HTTP methods applied to two URLs per resource, the **collection** and the **item**.

The endpoints of a resource

CRUD	Method + path	Request body	Success	Typical failures
Read (list)	GET <code>/api/tasks</code>	none	200 + array	400 bad query value
Read (one)	GET <code>/api/tasks/:id</code>	none	200 + the item	404
Create	POST <code>/api/tasks</code>	the new item (no id)	201 + Location + the item	400
Update (replace)	PUT <code>/api/tasks/:id</code>	the complete item	200 + the item	400, 404
Update (partial)	PATCH <code>/api/tasks/:id</code>	only the changed fields	200 + the item	400, 404
Delete	DELETE <code>/api/tasks/:id</code>	none	204, no body	404

```
// routes/books.js, mounted in app.js with app.use('/api/books', router)
// GET    /api/books      200 [ ... ]
router.get('/', listBooks);
// POST   /api/books      201 + Location
router.post('/', createBook);
// GET    /api/books/:id  200 | 404
router.get('/:id', getBook);
// PUT    /api/books/:id  200 | 400 | 404
router.put('/:id', replaceBook);
// PATCH  /api/books/:id  200 | 400 | 404
router.patch('/:id', patchBook);
// DELETE /api/books/:id  204 | 404
router.delete('/:id', deleteBook);
```

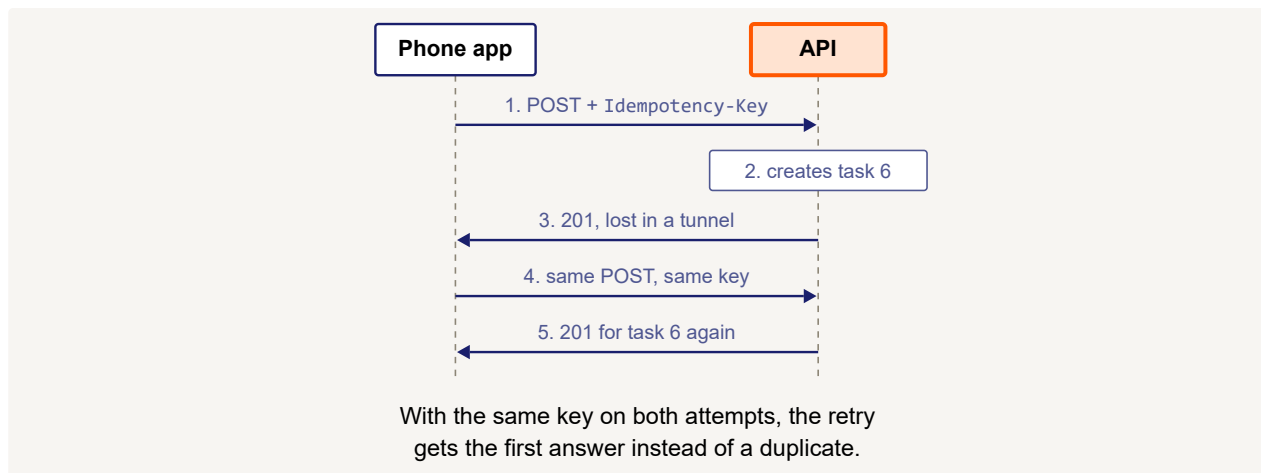
Example. One lifecycle in the request builder: GET `/api/tasks` lists five tasks (200); POST `/api/tasks` with {"title":"Study REST"} answers 201 and Location: `/api/tasks/6`; GET `/api/tasks/6` reads it (200); PATCH `/api/tasks/6` with {"done":true} changes one field (200); DELETE `/api/tasks/6` removes it (204); a last GET `/api/tasks/6` answers 404. One URL pattern, and the status codes tell the whole story.

Common mistake. Sending updates to the collection, such as PUT `/api/tasks` with {"id":6, "done":true} in the body. The URL must name the resource being changed, so the id belongs in the path: PATCH `/api/tasks/6`. On a well-designed API the collection version answers 405.

Safe and idempotent: why retries matter

A **safe** method changes nothing on the server (GET); an **idempotent** method leaves the server in the same state whether it is sent once or many times (GET, PUT, DELETE); POST is neither, and PATCH is not guaranteed

to be.



Retrying after a timeout

Method	Safe	Idempotent	A retry...
GET	yes	yes	is harmless
PUT	no	yes	sets the same full version again
DELETE	no	yes	leaves it deleted (the retry may answer 404)
PATCH	no	not guaranteed	is fine for "set a field to X", not for "add 1"
POST	no	no	may create a duplicate, unless an idempotency key is used

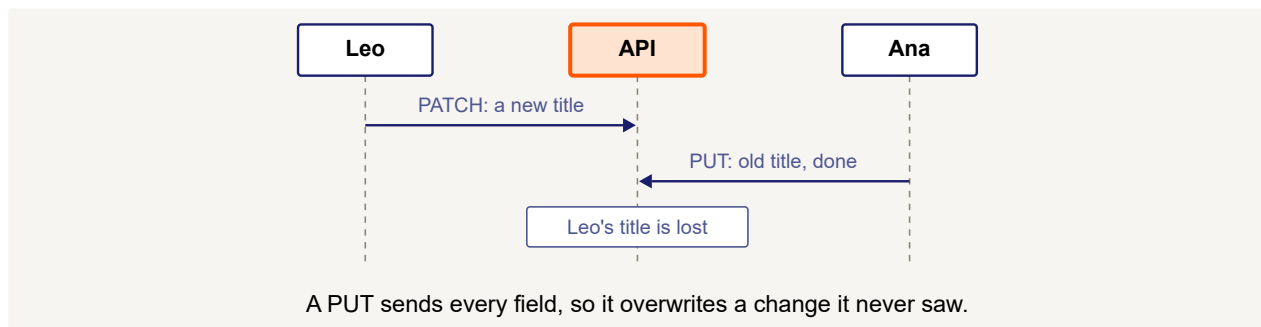
- **Idempotent is about the server's state**, not the response: the second DELETE /api/tasks/5 answers 404, but task 5 is gone either way.
- **An idempotency key** makes a POST safe to retry: the client invents a unique id per operation and sends it in a header (Idempotency-Key: 9b2e...); the server remembers the keys it has processed and answers a repeated key with the stored first response. Payment APIs work this way.
- **Clients know it too**: browsers ask "Confirm form resubmission?" before repeating a POST, and some HTTP libraries retry idempotent requests automatically but never POST.

Example. The train app sends POST /api/tasks with {"title":"Buy tickets"}, gets no answer and retries: without a key, the list now has two "Buy tickets". With Idempotency-Key: 9b2e... on both attempts, the server recognises the key and answers the same 201 Created with Location: /api/tasks/6, without creating task 7.

Common mistake. "DELETE is not idempotent, because the second call gets a 404 instead of a 204." Idempotence promises the same **effect on the server**, not the same status code. The response may differ; the state (the task is gone) does not.

PUT replaces, PATCH changes

PUT sends the **complete** new version of a resource and replaces it (fields left out go back to their defaults, or the request is rejected); **PATCH** sends **only the fields to change** and keeps all the others.



PUT vs PATCH

	PUT	PATCH
The body	The complete item	Only the fields to change (a plain JSON object, formally a "merge patch")
Fields left out	Back to their defaults; a missing required one is a 400	Kept as they are
Validation	Full mode: every required field	Partial mode: only the fields present; a body with none is a 400
Idempotent	Yes	Not guaranteed

```
const task = { id: 2, title: 'Add full CRUD', done: false, userId: 1 };
// what the client sent
const body = { done: true };

// PUT: build the whole new version from the body (+ defaults)
const afterPut = {
  id: task.id,
  title: body.title,
  // → undefined → a PUT without title must be a 400
  done: body.done ?? false,
  userId: body.userId ?? null,
};
console.log('PUT ', afterPut);

// PATCH: keep everything, overwrite only the allowed fields that were sent
const afterPatch = { ...task };
for (const key of ['title', 'done', 'userId']) {
  if (key in body) afterPatch[key] = body[key];
}
console.log('PATCH', afterPatch);
```

Example. Task 2 is { "title": "Add full CRUD", "done": false, "userId": 1 }. Leo renames it with PATCH /api/tasks/2 and {"title":"Add CRUD + tests"}. A second later Ana, whose screen still shows the old title, ticks it with PUT /api/tasks/2 and {"title":"Add full CRUD", "done":true, "userId":1}. Leo's title is gone. Had Ana sent PATCH with {"done":true}, both changes would be kept.

Common mistake. Writing PUT as a merge, such as `Object.assign(task, req.body)`. That is PATCH behaviour (fields not sent are kept), so the API has no real PUT, and the body can also overwrite id or add fields you never meant to store. A PUT builds the new object from the allowed fields only.

Status codes and errors

Success codes: 200, 201 and 204

A successful call answers **200 OK** with a body, **201 Created** with a `Location` header after something was created, or **204 No Content** when there is nothing to send back.

Which success code

Situation	Code	Body	Header to add
GET a list or one item	200 OK	the data	X-Total-Count for paged lists
POST created something	201 Created	the new resource	Location: /api/tasks/6
PUT / PATCH updated	200 OK	the resource as it is now	
DELETE removed	204 No Content	none	
List or search with no matches	200 OK	[]	

201 Created Goes with `Location`, the URL of the new resource (relative is fine). The body echoes it, including the fields the server set (id, createdAt), so the client needs no extra GET.

204 No Content Has **no body at all**: Express drops one if you pass it, and `await res.json()` on a 204 throws "Unexpected end of JSON input".

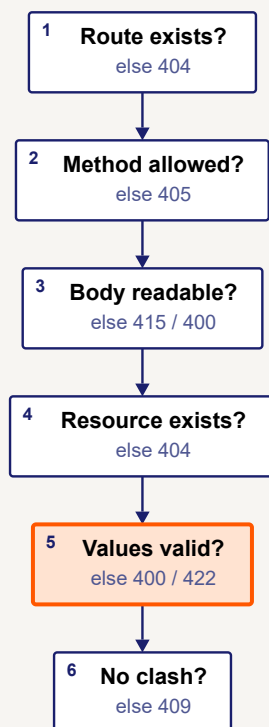
```
res.json(books);
// → 200 is the default status
// created
res.status(201).location(`/api/books/${book.id}`).json(book);
res.json(updatedBook);
// → 200 after PUT / PATCH
// deleted, no body
res.status(204).end();
```

Example. With curl (see [curl in five flags](#)), `curl -i -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d '{"title":"Study"}'` prints HTTP/1.1 201 Created and Location: /api/tasks/6 before the body: the client already knows everything; the JSON adds the id and createdAt the server chose.

Common mistake. Writing `res.status(204).json({ message: "Deleted" })` and expecting the client to show the message. A 204 never has a body: Express silently drops it. Either answer 204 with nothing, or 200 with a body.

Client errors: choosing the right 4xx

A **4xx** code tells the caller that **its request** must change, and each code names a different problem: unreadable or invalid (400), no such resource (404), wrong method (405), a clash with stored data (409), a body in the wrong format (415), or readable data that breaks a rule (422, where an API uses it).



The first check that fails decides the code.

The 4xx codes of a REST API

Code	Typical case	What the client must fix
400 Bad Request	Malformed JSON; a missing title (if the API uses 400 for validation); <code>?limit=abc</code>	The syntax or the values
404 Not Found	<code>GET /api/tasks/999; /api/taks</code>	The id or the path
405 Method Not Allowed	<code>DELETE /api/tasks</code> (the whole collection)	Use a method from the Allow header
406 Not Acceptable	<code>Accept: text/html</code> on a JSON-only API	The Accept header (the format it <i>asks for</i>)
409 Conflict	Fine on its own but clashes with stored data: an email already registered, deleting something other data points to	Different data, or change the state first
415 Unsupported Media Type	A JSON body sent without Content-Type: <code>application/json</code>	The Content-Type header (the format it <i>sends</i>)

Code	Typical case	What the client must fix
422 Unprocessable Content	<code>{"title":""}</code> on an API that keeps 400 for parse errors	The values

400 or 422? Many APIs, including the examples here, use 400 for both "cannot read it" and "the values break a rule". Others keep 422 for the second. Both are fine; follow the one your API documents, and never mix them.

404 or 405? 404: the path names nothing. 405: the path exists, but not with this method. Express answers 404 for an unmatched method unless you add a fallback, as in the code sample.

```
// router.route(path) attaches several methods to one path; .all() catches the rest
router.route('/')
  .get(listBooks)
  .post(createBook)
  .all((req, res) => res.set('Allow', 'GET, POST').status(405).json({ error: 'Method not allowed' }));
```

Example. Five versions of a sign-up, `POST /api/users`: with the body in plain text, **415**; with `{"name":"Iris",}` (a trailing comma), **400** (cannot parse); with `{"name":"","email":"nope"}`, **400** with two field errors (or 422 elsewhere); with a valid body but an email that already exists, **409**; sent as `PATCH /api/users`, **405** with `Allow: GET, POST`.

Common mistake. Answering **500** for bad input. If invalid data reaches code that assumes it is valid, `req.body.title.trim()` throws "Cannot read properties of undefined" and the error handler sends 500, telling the client the **server** failed. Validate first and answer 400: the caller made the mistake and can fix it.

401, 403 or 404: who are you, and may you?

401 means the server does not know who you are, **403** means it knows and you are not allowed, and some APIs answer **404** instead of 403 so as not to reveal that a resource exists.

Identity and permission codes

Code	When	What the client does
401 Unauthorized	No token, a malformed one or an expired one. Despite its name, it is about <i>identity</i> . It comes with <code>WWW-Authenticate: Bearer</code> .	Logs in (or refreshes the token) and tries again
403 Forbidden	A known identity that may not do this	Nothing: the same request will never work
404 Not Found	Another user's private resource: 403 would confirm that it exists	Treats it as missing

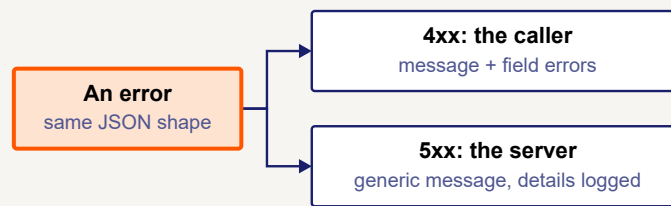
Example. In the request builder, `POST /api/users` is for admins. With no Authorization header: **401** with `WWW-Authenticate: Bearer`. With `Authorization: Bearer nonsense`: **401** again (an unknown token is no identity). With `Authorization: Bearer student-token`: **403** ("you are a student"). With `Authorization: Bearer admin-token` and a valid body: **201**.

Common mistake. Sending 401 for "logged in but not allowed". The front end reacts to 401 by sending the user to the log-in page; they log in again, get 401 again, and loop forever. "Not allowed" is 403.

One error shape, and when to use 500

Every error answer should have the **right status code** and the **same JSON shape**, for example `{ "error": "message" }` **plus an errors list** of `{ field, message }` for validation, so a client handles every error with one piece

of code; **500** is only for failures of the server itself.



The caller's mistakes are explained; the server's failures are only logged.

error Always present: a short human-readable message, such as "Task 99 not found".

errors Added when there are per-field problems, listing **all** of them: [{ "field": "title", "message": "title is required" }] (see [Validating input](#)).

```
// app.js: the LAST middleware. Every error leaves in the same shape.
app.use((err, req, res, next) => {
  const status = err.status || 500;
  // details stay in the server log
  if (status >= 500) console.error(err);
  res.status(status).json({
    error: status >= 500 ? 'Internal server error' : err.message,
    // field errors, when there are any
    ...(err.errors ? { errors: err.errors } : {}),
  });
});
```

Example. Because the shape never changes, the client needs one function: `const res = await fetch(url, options); if (!res.ok) { const { error, errors = [] } = await res.json(); errors.forEach((e) => showFieldError(e.field, e.message)); showBanner(error); }`. It works for a 400 with field errors, a 404 and a 500 alike.

Common mistake. Answering `res.status(200).json({ success: false, error: "Not found" })`. Every tool that reads status codes now records a success, and `res.ok` is true in the browser. Put the meaning in the status code (404) and keep the body for the details.

The API contract

JSON in, JSON out: Content-Type and Accept

A JSON API reads a request body only when its Content-Type: application/json header says it is JSON, always answers with Content-Type: application/json, and can read the client's Accept header to see which formats it understands.

Conventions that make an API predictable

- **Keys in camelCase:** `createdAt`, the JavaScript habit.
- **Dates as ISO 8601 strings in UTC:** `"2026-10-05T10:00:00.000Z"`, because JSON has no date type.
- **Real booleans:** `true`, never `"true"` or `"yes"`.
- **null for "no value",** always the same way.
- **A JSON object** at the top level of a request body.
- **Strict answers:** **415** to a body that is not JSON, **406** when Accept rules JSON out. The request builder does both.

```
const book = { title: 'Dune', read: false, due: new Date(Date.UTC(2026, 9, 20)), notes: undefined };

// the Date becomes an ISO string; undefined disappears
console.log(JSON.stringify(book));
// what fetch sends if you forget JSON.stringify
```

```

console.log(String({ title: 'Dune' }));

const back = JSON.parse('{"title":"Dune","due":"2026-10-20T00:00:00.000Z"}');
// a string: JSON has no date type
console.log(typeof back.due);

try {
  // single quotes are not JSON
  JSON.parse('{title: 'Dune'}');
} catch (e) {
  console.log('400 material:', e.name);
}

```

Example. The right fetch call: `fetch("/api/books", { method: "POST", headers: { "Content-Type": "application/json", Accept: "application/json" }, body: JSON.stringify(book) })`. Leave out the header and an Express 4 API sees `req.body` as `{}`; leave out `JSON.stringify` and the body is the text `[object Object]`, which no JSON parser can read.

Common mistake. Debugging the validation code when the real problem is a missing Content-Type header. If the API says a field is missing that you can see in your request, check the header first: `curl -i` (or the request builder) shows exactly what was sent.

The API contract vs the stored record (DTOs)

A **DTO** (Data Transfer Object) is the shape of the data your API sends and accepts. It is deliberately different from what the server stores: it leaves out secrets and internal fields, and uses the JSON naming convention.



Only the fields on the allow-list reach the client.

- Secrets** password_hash: the stored, one-way scrambled form of a password (see [Hashing, not encryption](#)). It must never leave the server.
- Internal fields** A **soft-delete** flag such as `is_deleted` (the record is kept but treated as gone), internal notes, references to other records.
- Naming** Databases usually name fields in **snake_case** (`created_at`); JSON uses **camelCase** (`createdAt`). The mapping renames them (the SQL side is in [From rows to JSON](#)).

- **Map with an allow-list:** copy the fields you want (`toUserDto(record)`), never delete the ones you remember. A field added tomorrow cannot leak by accident.
- **The same on the way in:** copy only the fields a client may set. Otherwise a sign-up body with `"role": "admin"` makes the caller an admin: **mass assignment**.

```

// One row as the database returns it
const row = { id: 2, email: 'leo@example.com', password_hash: '$2b$10$Qm1x...',
  role: 'student', created_at: '2026-09-02T08:00:00.000Z', is_deleted: false };

// X Block-list: remove what you remembered to remove
const { password_hash, ...leaky } = row;
// is_deleted and created_at leak, and so will any new column
console.log(leaky);

// ✓ Allow-list: copy only the contract
const toUserDto = (r) => ({ id: r.id, email: r.email, role: r.role, createdAt: r.createdAt });
console.log(toUserDto(row));

```

Example. In the request builder, the server state panel shows the stored user rows with `password_hash`, `created_at` and `is_deleted`. Send `GET /api/users`: every user comes back as `{ "id", "name", "email", "role", "createdAt" }`. Create a user with a password: the 201 body contains neither the password nor its hash.

Common mistake. Sending the record straight out, `res.json(user)`, or fixing it with `delete user.password_hash`. The first leaks the hash; the second is a block-list that forgets the next secret field (and, on a shared object, deletes the hash from your data too).

Building and testing

CRUD with an in-memory array

Before a database, an API can keep its data in a JavaScript **array** in memory: each handler finds the item by its id, answers **404** if it is missing, and changes the array in place, behind function names that a database layer can keep later.

- 1. Turn the id into a number:** route parameters are strings (see [Route parameters](#)), and `"2" === 2` is false.
- 2. Find it:** `find` when you only need the item, `findIndex` when you must replace or remove it.
- 3. Missing? Return early:** `return res.status(404).json(...)`. Without `return` the handler answers twice (see [Exactly one response per request](#)).
- 4. Build new objects field by field** from what the client may set.

```
let books = [{ id: 1, title: 'Dune', read: true }];
let nextId = 2;

function getBook(req, res) {
  const id = Number(req.params.id);
  // → "1" → 1
  const book = books.find((b) => b.id === id);
  if (!book) return res.status(404).json({ error: `Book ${id} not found` });
  res.json(book);
}

function deleteBook(req, res) {
  const i = books.findIndex((b) => b.id === Number(req.params.id));
  if (i === -1) return res.status(404).json({ error: 'Book not found' });
  books.splice(i, 1);
  res.status(204).end();
}
```

```
const books = [{ id: 1, title: 'Dune' }];
console.log(books.find((b) => b.id === '1'));
// → undefined: "1" is not 1
// found
console.log(books.find((b) => b.id === Number('1')));

let nextId = 2;
// what a client could send
const body = { id: 1, title: 'Hacked', isAdmin: true };
// X the body overwrote the id
console.log({ id: nextId++, ...body });
// ✓ only the allowed fields
console.log({ id: nextId++, title: body.title });
```

Example. DELETE `/api/books/7` when book 7 does not exist: `findIndex` returns `-1`, the handler answers 404 and returns. Without that check, `books.splice(-1, 1)` would silently remove the **last** book and answer 204: the wrong item deleted, and a success reported.

Common mistake. Forgetting `return` before an early answer: `if (!book) res.status(404).json(...)` followed by `res.json(book)`. The second call crashes with `ERR_HTTP_HEADERS_SENT`. Every early answer starts with `return`.

Validating input: 400 with field errors

Validation checks every field of the request body (present, right type, right length or range, allowed value) **before** anything is stored, and rejects a bad request with **400** and a list of **all** the problems, each tied to its field.



Bad input stops at the boundary; only clean data reaches the controller.

Required	Present, and not empty after <code>trim()</code> .
Type	<code>typeof v === "string"</code> ; <code>typeof v === "boolean"</code> (the string "true" is not a boolean); <code>Number.isInteger</code> for whole numbers.
Length or range	A title of at most 120 characters, a rating from 1 to 5.
Closed set	<code>["easy", "medium", "hard"].includes(v)</code> .
Unknown fields and id	Stripped and ignored.

- **Two modes:** full for POST and PUT (every required field must be there), partial for PATCH (only the fields present).
- **Collect every error** instead of stopping at the first, so the client shows all messages at once.
- **A pure function** (data in, { valid, errors } out, no req or res) can be tested without Express. Schema libraries such as Zod do the same with less code (see [Validation with a schema](#)).

```

function validateBook(body, { partial = false } = {}) {
  const errors = [];
  const has = (k) => body[k] !== undefined;
  if (!partial || has('title')) {
    if (typeof body.title !== 'string' || !body.title.trim()) {
      errors.push({ field: 'title', message: 'title is required' });
    }
  }
  if (has('rating') && !(Number.isInteger(body.rating) && body.rating >= 1 && body.rating <= 5)) {
    errors.push({ field: 'rating', message: 'rating must be a whole number from 1 to 5' });
  }
  return { valid: errors.length === 0, errors };
}

console.log(validateBook({ title: 'Dune', rating: 5 }));
// two errors, both reported
console.log(validateBook({ title: ' ', rating: '5' }));
// PATCH: title not needed
console.log(validateBook({ rating: 4 }, { partial: true }));
  
```

Example. POST /api/books with {"title": " ", "rating": "5"} answers 400 with { "error": "Validation failed", "errors": [{ "field": "title", "message": "title is required" }, { "field": "rating", "message": "rating must be a whole number from 1 to 5" }] }, and the collection has the same length as before. The client shows each message under its input.

Common mistake. Checking only `if (!req.body.title)`. It accepts "title": 123 and "title": " ", says nothing about the other fields, and reports one problem at a time, so the user fixes, resubmits and gets the next error, again and again.

Testing by hand: curl, Postman, Thunder Client

Testing an API by hand means sending each request yourself and reading the raw answer, status and headers included: with **curl** in a terminal, or with a GUI client such as Postman or Thunder Client that saves requests in collections.

- **Always -i:** without it you see only the body, and a 201 looks like a 200.
- **A JSON body needs its header:** -d alone labels the body as a form, so add -H "Content-Type: application/json".
- **Quote URLs with &:** unquoted, the shell reads & as "run in the background".
- **Not sure what was sent?** -v shows the request too.

On Windows

Windows PowerShell 5.1

curl is an alias for another command (Invoke-WebRequest): type curl.exe. Even single quotes are not enough for JSON there: -d '{\"title\": \"Study\"}'.

cmd.exe

Write -d \"{\\\"title\\\":\\\"Study\\\"}\".

Any shell

Put the JSON in a file and send -d @task.json (in PowerShell, quote it: "@task.json"). The single-quoted JSON in the code sample works in Git Bash, macOS and Linux.

shell (Git Bash, macOS, Linux)

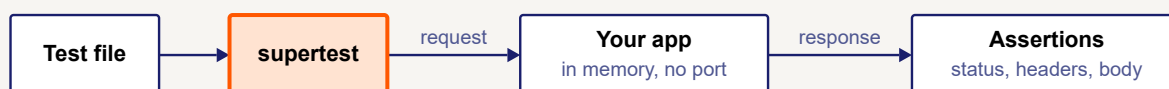
```
curl -i http://localhost:3000/api/tasks
curl -i "http://localhost:3000/api/tasks?done=false&sort=createdAt&order=desc"
curl -i -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Study REST"}'
curl -i -X PATCH http://localhost:3000/api/tasks/2 \
  -H "Content-Type: application/json" -d '{"done":true}'
curl -i -X DELETE http://localhost:3000/api/tasks/2
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:3000/api/me
```

Example. The POST in the code sample prints HTTP/1.1 201 Created, then headers including Location: /api/tasks/6 and Content-Type: application/json; charset=utf-8, an empty line, and the JSON body. That is the whole contract of a create; without -i you would only see the JSON and could not tell 201 from 200.

Common mistake. Typing curl -X POST ... -H "Content-Type: application/json" in Windows PowerShell 5.1 and getting "Cannot bind parameter Headers": that is Invoke-WebRequest speaking, not curl. Use curl.exe, or Git Bash.

Automated API tests with supertest

An automated API test sends a request to your Express app **inside the test process with supertest**, then checks the status code, the body and, for writes, the change in state; a test runner (Jest, or Node's built-in node:test) runs them all with npm test.



No port, no browser: the request goes straight into the app.

Test runner Finds the test files, runs them and reports what passed (Jest: npm test runs jest).

Test One case: test("...", async () => { ... }); describe groups several.

Assertion Checks one fact and fails the test if it is false:
 expect(res.status).toBe(201). toBe compares with ===, toEqual compares contents, toMatchObject checks the body contains **at least** the listed fields (ideal when the server adds an id). With node:test:
 assert.equal(res.status, 201).

supertest `request(app).post("/api/books").send({ title: "Dune" })` sends the request and resolves to the response (`res.status`, `res.headers.location`, `res.body`). `.send(object)` sets the JSON header; `.set("Authorization", "Bearer ...")` adds any header. It needs the app exported without listening: an `app.js` that calls `app.listen()` opens a real port in every test file (see [app.js builds](#), [server.js starts](#)).

- **Assert four things:** the status, the important headers (`Location`), the body shape and, for writes, the state change: after a create a GET finds it; after a delete a GET gives 404; after a rejected POST the list has the same length.
- **Keep tests independent:** the in-memory array survives from one test to the next, so reset the store before each test (`beforeEach`) or let each test create the data it needs.

```
// tests/books.test.js (Jest + supertest)
const request = require('supertest');
// the app, NOT server.js
const app = require('../src/app');

test('POST /api/books → 201 with Location and a server id', async () => {
  const res = await request(app).post('/api/books').send({ title: 'Dune' });
  expect(res.status).toBe(201);
  expect(res.body).toMatchObject({ title: 'Dune' });
  expect(res.headers.location).toBe(`/api/books/${res.body.id}`);
});

test('POST without a title → 400, and nothing is added', async () => {
  const before = await request(app).get('/api/books');
  const res = await request(app).post('/api/books').send({});
  expect(res.status).toBe(400);
  const after = await request(app).get('/api/books');
  expect(after.body.length).toBe(before.body.length);
});
```

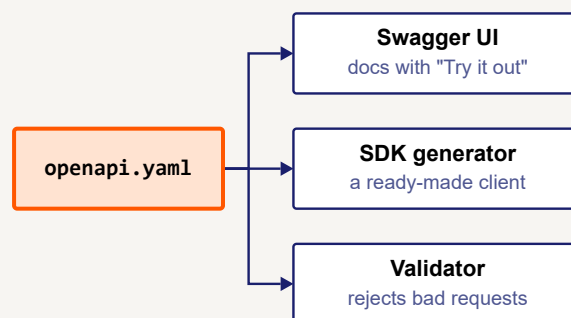
Example. Run `npm test` and Jest prints one line per test, ✓ or ✗. A failure shows Expected: 201, Received: 200: the create handler is missing `res.status(201)`. You fix one line, run again, and every other route is checked again for free.

Common mistake. Tests that depend on each other. The first test creates book 2, and the second assumes the list has two books: run the second alone, or in another order, and it fails although the API is fine. Reset the store in `beforeEach`, so every test starts from the same data.

Documenting and other API styles

Documenting the API: OpenAPI

An **OpenAPI** document (formerly called Swagger) describes every endpoint of an API in one YAML or JSON file: paths, methods, parameters, request and response bodies and status codes, in a form that people and programs can both read.



Written once, the spec feeds people, tools and generators.

info	Title and version.
paths	Each path, then each method, with its parameters, requestBody and responses per status code.
components.schemas	The shapes of your DTOs, written once and referenced with \$ref.

- **Keep it in the repository** and change it in the **same commit** as the code, or it stops being true within a week.
- **The minimum for a small project:** an endpoint table in the README (method, path, query parameters, success status, error statuses, an example request and response). An `openapi.yaml` holds the same facts.

YAML

```
openapi: 3.0.3
info: { title: Books API, version: 1.0.0 }
paths:
  /api/books:
    post:
      summary: Add a book
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/NewBook' }
      responses:
        '201':
          description: Created. Location holds the URL of the new book.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Book' }
        '400': { description: Validation failed (error + errors list) }
  /api/books/{id}:
    get:
      parameters:
        - { in: path, name: id, required: true, schema: { type: integer } }
      responses:
        '200': { description: OK }
        '404': { description: No book with that id }
components:
  schemas:
    NewBook:
      type: object
      required: [title]
      properties:
        title: { type: string, minLength: 1, maxLength: 120 }
        read: { type: boolean, default: false }
    Book:
      allOf:
        - $ref: '#/components/schemas/NewBook'
        - properties:
            id: { type: integer }
            createdAt: { type: string, format: date-time }
```

Example. A front-end developer asks: "what do I get if the title is missing?" With the spec they open `/api/books` → post → responses and see 400, "Validation failed (error + errors list)", plus the `NewBook` schema saying that `title` is required and at most 120 characters long. In Express, the `swagger-ui-express` package serves the same file as a page at `/docs`.

Common mistake. Writing the documentation at the end and never touching it again. Documentation that disagrees with the API is worse than none, because people trust it. Update the spec, or the README table, in the same commit as each change to a route.

REST, GraphQL and other API styles

GraphQL is an alternative to REST in which the client sends a **typed query** to one endpoint (`/graphql`) and gets exactly the fields it asked for; REST has many URLs, each with a fixed response shape.

REST vs GraphQL

	REST	GraphQL
Endpoints	Many URLs, one per resource	One URL, usually <code>POST /graphql</code>
Response shape	Fixed per endpoint: the list screen gets every field (over-fetching)	Exactly the fields asked for
One screen, several resources	Several requests in a row (under-fetching)	One query, one round trip
Describing the API	OpenAPI, optional	A typed schema , required
Caching	By URL, for free	Not by URL: everything goes to one
Errors	The status code says what happened	Usually 200 OK with an <code>errors</code> array in the body

gRPC Compact binary messages over HTTP/2 with typed contracts: fast calls between a company's own back-end services.

WebSockets One connection that stays open, where both sides can send: chat, live notifications, games.

Webhooks The server calls **your** URL when something happens: "tell me when the payment is completed".

GraphQL

```
# One request: POST /graphql
query {
  user(id: 2) {
    name
    tasks(done: false) { title }
  }
}

# The answer (status 200 OK, even when part of it failed)
{ "data": { "user": { "name": "Leo Martín",
  "tasks": [ { "title": "Return the right status codes" } ] } } }
```

Example. A dashboard shows a user's name and the titles of their unfinished tasks. REST: `GET /api/users/2` and `GET /api/users/2/tasks?done=false`, two requests, each returning all fields. GraphQL: the single query in the code sample, returning only name and title. For a simple client (one list, one resource) the REST version is simpler, and the browser can cache it.

Common mistake. Thinking GraphQL is a database, or "REST but newer". It is a different way to shape an API, with its own costs; most projects start with REST and add GraphQL only where flexible queries pay off.